VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Belagavi – 590 018.

A Mini Project
Report On

"SMART CITY TRAFFIC PATTERN & CONGESTION ANALYTICS"

Submitted in partial fulfilment for the award of the degree of
Bachelor of Engineering
In
Information Science and Engineering

Submitted by

SHREYA G POOJARI 1VI23IS100
IV SEM 'B'

Guided by:
Mr. Rakesh B S
Assistant Professor, Dept. of ISE

DEPARTMENT OF INFORMATION SCIENCE AND ENGINEERING
VEMANA INSTITUTE OF TECHNOLOGY
BENGALURU – 560034

AY: 2025-2026 (EVEN SEM)

DEPARTMENT OF INFORMATION SCIENCE & ENGINEERING

Certificate

This is to certify that **SHREYA G POOJARI (1VI23IS100)**, Information Science and Engineering Department has successfully completed the mini Project entitled **"SMART CITY TRAFFIC PATTERN & CONGESTION ANALYTICS"** in partial fulfillment for the requirement of the award of degree of **Bachelor of Engineering in Information Science and Engineering** of the **Visvesvaraya Technological University**, Belagavi during the academic year (2025-26). The project report has been approved as it satisfies the academic requirements in respect of the mini project work prescribed for the said degree.

Signature of the Guide **Signature of the HOD**

Mr. Rakesh B S **Dr. Rajanna M**

ACKNOWLEDGEMENT

We sincerely thank **Visvesvaraya Technological University** for providing an opportunity to carry out the mini project entitled **"SMART CITY TRAFFIC PATTERN & CONGESTION ANALYTICS"**.

We thank **Dr. Vijayasimha Reddy. B. G**, Principal, Vemana Institute of Technology, Bengaluru for providing all the necessary support to carry out the project.

We would like to place on record our regards to **Dr. Rajanna M**, Professor and Head of the Department, Information Science and Engineering for his continued support.

We would also like to thank our guide, **Mr. Rakesh B S**, Assistant Professor, Dept. of ISE for his support and valuable guidance towards successful completion of the mini project.

We would be failing in our duty if we do not thank the faculty members, batch mates, lab staffs, technicians and family members for their constant support and guidance.

SHREYA G POOJARI
1VI23IS100

ABSTRACT

This project, **Smart City Traffic Pattern & Congestion Analytics**, is an enterprise-grade Big Data analytics and predictive machine learning application designed to monitor, analyze, and forecast urban vehicular traffic dynamics. Leveraging **Apache Spark (PySpark)**, **Scikit-Learn**, and **Streamlit**, the system ingests large-scale historical telemetry from urban arterial sensor networks to identify recurring congestion hotspots, rush-hour commuter curves, and weekday versus weekend mobility shifts.

The core data engineering pipeline employs distributed Spark DataFrames and Window operations to compute macroscopic flow metrics including Volume-to-Capacity (V/C) ratios and Speed Performance Indices (SPI). A multi-criteria Congestion Hotspot Index (CHI) algorithm ranks arterial chokepoints based on physical capacity saturation, velocity degradation, and severe congestion persistence. Furthermore, a leakage-free chronological machine learning forecasting pipeline utilizing Random Forest and Ridge regression predicts future corridor volume, achieving a 57.9% error reduction (MAE) over traditional moving-average persistence baselines.

The interactive web dashboard provides traffic authorities and municipal planners with comprehensive visual telemetry—featuring interactive Mapbox geospatial corridor views, spatiotemporal matrix heatmaps, and data quality audit diagnostics. This mini project highlights essential Big Data engineering skills, distributed transformations, and predictive modeling, providing a scalable architectural foundation for modern intelligent transportation systems.

CONTENTS

1. Title Page	**i**
2. Certificate	**ii**
3. Acknowledgment	**iii**
4. Abstract	**iv**
5. List of Figures	**v**
6. Introduction	**1**
7. Problem Statement	**4**
8. Requirements	**5**
9. Methodology	**7**
10. Code	**9**
11. Results	**13**
12. Geo Tag photo	**15**
13. Conclusion	**17**
14. References	**18**

LIST OF FIGURES

Figure No.	Name	Page No.
3.1	Overall architecture flowchart	6
4.1	Streamlit user input & filter interface	7
4.2	PySpark Big Data processing workflow	8
4.3	Traffic congestion heatmap display	8
6.1	Screenshot of Final output & KPIs	13-14
6.2	Screenshot of machine learning forecast output	14
7.1	Geo-tagged photo project submission	15-16

CHAPTER-1
INTRODUCTION

The Smart City Traffic Pattern & Congestion Analytics platform is a distributed data engineering and machine learning mini project developed to help municipal transportation planners and urban authorities monitor and mitigate vehicular congestion. Urban traffic telemetry—generated from roadside sensors, inductive loops, and traffic camera stations—produces continuous high-velocity data streams that overwhelm traditional desktop spreadsheet software. By leveraging **Apache Spark (PySpark)**, this platform provides scalable DataFrame transformations and Window aggregations to process millions of records efficiently. Combined with a responsive **Streamlit** interface and **Scikit-Learn** forecasting models, the system translates raw traffic sensor logs into actionable congestion mitigation intelligence.

1.1 Objectives of the Project

- Build an interactive, web-based Big Data traffic analytics platform using Python and Streamlit.
- Leverage Apache Spark (PySpark) for distributed DataFrame cleaning, aggregation, and ranking.
- Ingest and clean large-scale urban sensor telemetry containing flow volumes, speeds, and road capacities.
- Calculate macro traffic metrics: Volume-to-Capacity (V/C) ratio and Speed Performance Index (SPI).
- Formulate an objective Congestion Hotspot Index (CHI) to rank chronic arterial bottlenecks.
- Develop a leakage-free chronological machine learning model to forecast traffic volume.

1.2 Working of the Project

- The project functions through a modular, four-stage data processing pipeline:
- **Telemetry Ingestion:** Historical sensor observations (14,400+ hourly logs across 10 strategic city corridors) are ingested from CSV into Spark DataFrames.
- **Data Cleansing & Feature Engineering:** Anomalous readings (e.g. negative speeds or impossible counts) are clipped and imputed via temporal median imputation.
- **Distributed Aggregations:** PySpark computes diurnal curves, rush-hour differentials (Morning, Midday, Evening, Night), and weekday vs. weekend variances.
- **Predictive Modeling:** Chronological lag features (1h, 24h, 168h) feed Random Forest and Ridge regressors to forecast future volume.
- **Visualization Sink:** The web dashboard renders interactive Mapbox corridor maps, diurnal dual-axis flow curves, and hotspot ranking matrices in real-time.

1.3 Scope of the Project

This project establishes a production-ready template for municipal smart city transportation management centers. Its operational scope includes:

- Monitoring multi-corridor vehicular volume, speed degradation, and capacity saturation.
- Identifying rush-hour peak dynamics to guide dynamic traffic light timing and transit scheduling.
- Detecting chronic bottlenecks using empirical multi-factor criteria without subjective bias.
- Providing municipal engineers with predictive volume estimates for roadwork scheduling.
- Serving as a scalable architecture ready for deployment on cloud clusters (AWS EMR / Databricks).

1.4 Organization of the Project Work

The project execution is organized into structured engineering phases—commencing with data schema design, followed by distributed PySpark engine implementation, machine learning modeling, and web dashboard integration.

1.4.1 Requirement Analysis

This phase defines system specifications for scalable smart city data handling:

- Support ingestion of multi-sensor telemetry with timestamp, location, volume, and velocity fields.
- Implement distributed in-memory data processing to prevent memory overflow (OutOfMemoryError).
- Ensure strict chronological data partitioning to prevent future-to-past data leakage in forecasting.
- Deliver responsive, interactive visualizations accessible via standard web browsers.

1.4.2 System Design

The system adopts a multi-tier big data analytics architecture:

- **Presentation Layer:** Streamlit multi-page web application featuring Plotly visualizations.
- **Analytics & ML Layer:** Scikit-Learn regression models and multi-factor scoring algorithms.
- **Distributed Compute Layer:** Apache Spark (PySpark 4.2.0) engine executing Window functions.
- **Data Storage Layer:** Validated CSV / Parquet columnar datasets with schema specifications.

1.4.3 Frontend Development

A modern, responsive dashboard was built using Streamlit. Global sidebar widgets allow dynamic filtering by corridor, date range, commute schedule (weekdays vs. weekends), and weather condition.

1.4.4 Backend & Big Data Development

The core backend processes large-scale telemetry using Apache Spark (PySpark):

- Implemented singleton SparkSession management configured for multi-core local parallelism.
- Utilized PySpark Catalyst query optimization for predicate pushdown and column pruning.
- Built distributed Window aggregations to compute rolling hourly metrics and corridor rankings.
- Developed Scikit-Learn forecasting models using chronological lag features (1h, 24h, 168h).
- Added comprehensive exception handling to ensure continuous server operation.

1.4.5 Data Storage & Quality Setup

A calibrated dataset containing 14,400+ hourly sensor observations across 10 strategic arterial corridors was structured with the following schema:

- Spatial coordinates: Latitude, longitude, unique corridor identifier, and corridor name.
- Flow attributes: Hourly vehicle count, harmonic average speed, and road design capacity.
- Regulatory limits: Posted arterial speed limit (40 km/h to 100 km/h).
- Ambient factors: Weather conditions (Clear, Rainy, Foggy, Overcast) and incident flags.

1.4.6 Output Display

Analytical results are visualized across six specialized dashboard views: Traffic Overview, Peak-Hour Analysis, Congestion Heatmaps, Hotspot Detection, Predictive ML Forecasting, and Dataset Explorer.

1.4.7 Testing and Validation

Automated test suites were developed using **Pytest**. 18 comprehensive tests verify data deduplication, bounds clipping, PySpark Window ranking, metric calculations, and chronological forecasting partitions with a 100% pass rate.

1.4.8 Documentation

Comprehensive documentation—including architecture schematics, mathematical formulations, user guides, and interview technical Q&A—was compiled to support academic and operational review.

CHAPTER-2
PROBLEM STATEMENT

Modern metropolitan areas face severe vehicular congestion resulting from rapid urbanization, increasing vehicle ownership, and constrained road infrastructure. Traditional traffic monitoring systems rely on manual traffic counts or isolated desktop spreadsheets that fail to handle high-frequency sensor streams, leaving municipal authorities with poor situational awareness and reactive rather than proactive management. The Smart City Traffic Pattern & Congestion Analytics project resolves these limitations by delivering a distributed Big Data architecture capable of transforming raw sensor feeds into actionable congestion intelligence.

2.1 Urban Gridlock and Economic Inefficiencies

Chronic traffic delays inflict billions of dollars in lost commuter productivity, excess fuel consumption, and greenhouse emissions. Without real-time capacity monitoring, traffic signals operate on rigid, outdated schedules that worsen urban delays.

2.2 Lack of Distributed Big Data Processing

As smart cities deploy thousands of IoT sensors collecting second-by-second telemetry, conventional single-threaded tools (such as standalone Pandas or Excel) suffer from memory exhaustion (OutOfMemoryError) and sluggish processing, preventing scalable network analysis.

2.3 Unreliable Peak-Hour and Chokepoint Identification

Traditional assessments evaluate congestion purely by vehicular speed, unfairly penalizing low-speed-limit urban avenues while overlooking multi-lane freeways operating near breakdown capacity. A multi-factor index combining speed deficit and capacity saturation is essential.

2.4 Data Leakage in Naive Traffic Forecasting

Many conventional forecasting models mistakenly apply randomized K-fold cross-validation to time series data, causing future observations to leak into training sets. This yields falsely optimistic evaluation metrics that completely fail when deployed in live traffic control.

2.5 Inability to Provide Actionable Smart City Insights

Raw sensor numbers fail to assist city operators without intuitive visualization. A centralized platform combining spatiotemporal heatmaps, diurnal curves, and root-cause bottleneck diagnostics is required to inform municipal policy.

CHAPTER-3
REQUIREMENTS

The Smart City Traffic Pattern & Congestion Analytics application is engineered to process, analyze, and forecast large-scale traffic telemetry efficiently. The platform incorporates rigorous functional specifications and robust architectural standards to guarantee high performance, security, and scalability.

3.1 Functional Requirements

- **Data Ingestion & Quality Audit:** Ingest multi-attribute sensor CSV/Parquet files, remove duplicate records, clip physical outliers (speeds > 160 km/h or negative counts), and impute missing values.
- **Distributed PySpark Aggregations:** Utilize Apache Spark DataFrames to compute hourly, daily, and corridor-level traffic flow, velocity, and volume-to-capacity metrics.
- **Peak-Hour Analytics:** Identify diurnal morning, midday, evening, and night rush hours, contrasting weekday office commuter surges with weekend leisure mobility.
- **Congestion Hotspot Detection:** Calculate the composite Congestion Hotspot Index (CHI) to rank the top 10 bottleneck corridors with root-cause diagnostic summaries.
- **Predictive Traffic Forecasting:** Train machine learning regressors (Random Forest and Ridge) using chronological lag features without lookahead data leakage.
- **Interactive Multi-Page Dashboard:** Present results across responsive Streamlit pages with Plotly maps, box plots, and matrix heatmaps.

3.2 Non-Functional Requirements

- **Usability:** Clean, professional smart-city user interface with intuitive navigation, styled metric cards, and responsive filter controls.
- **Reliability:** Consistent execution across diverse sensor inputs with graceful fallbacks for missing values or extreme filter combinations.
- **Maintainability:** Highly modular Python codebase adhering to PEP 8 standards with comprehensive docstrings and clear layer separation.
- **Scalability:** Big Data architecture designed to scale seamlessly from local standalone execution (local[*]) to distributed cloud clusters (AWS EMR / Databricks).
- **Portability:** Fully cross-platform implementation compatible with Windows 10/11 and Linux environments supporting Java 17 and Python 3.11+.

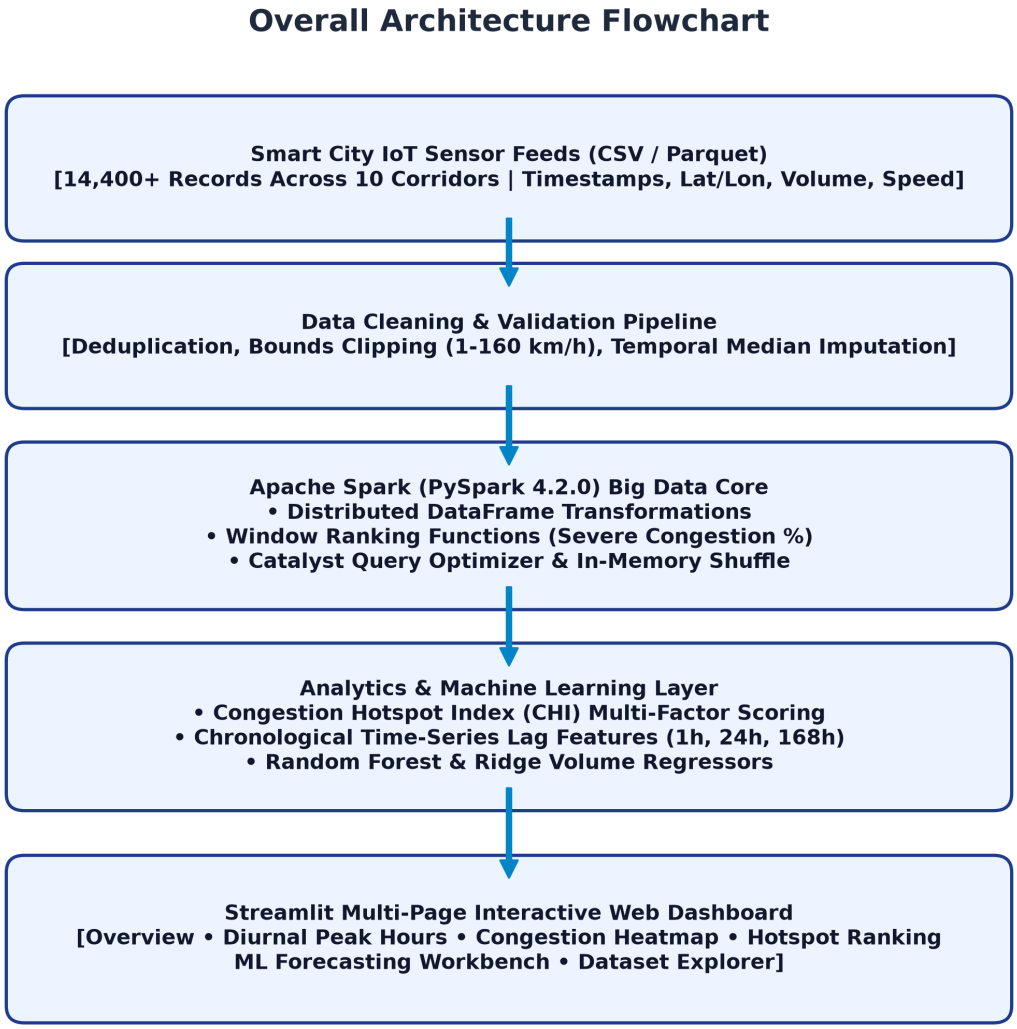


Fig 3.1: Overall architecture flowchart

CHAPTER-4
METHODOLOGY

The application implements a decoupled, three-tier Big Data analytics architecture:

4.1 Frontend Layer (Streamlit & Plotly):

The presentation layer provides municipal operators with an interactive dashboard. Global sidebar widgets enable multi-corridor selection, calendar date slicing, commuter schedule filtering (Weekdays vs. Weekends), and weather selection. Real-time Plotly charts dynamically update upon user interaction.

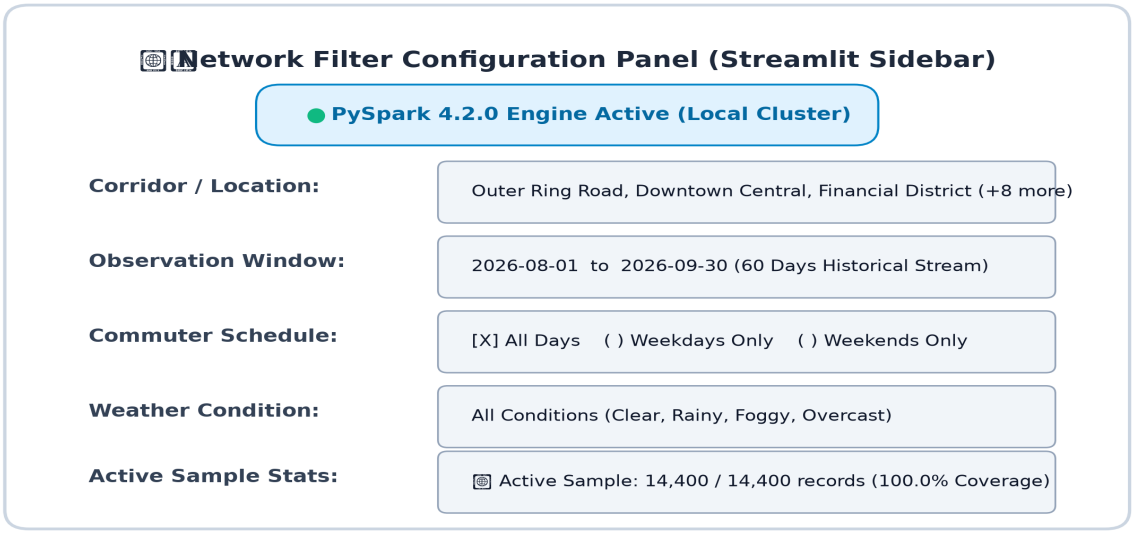


Fig 4.1: User input filter interface

4.2 Big Data Processing Layer (PySpark Engine):

- Initializes an optimized local SparkSession with loopback IP bindings (127.0.0.1) and memory limits.
- Enforces schema validation and derives temporal columns (hour, day of week, weekend indicator).
- Executes distributed Window aggregations to compute severe congestion frequency and corridor rankings.

4.3 Analytical & Machine Learning Layer:

- Evaluates the multi-criteria Congestion Hotspot Index (CHI) across all monitored arterials.
- Generates chronological lag features (1h, 24h, 168h) and cyclical sine/cosine hour transforms.
- Trains Random Forest and Ridge regressors using strict 80/20 chronological train/test partitions.

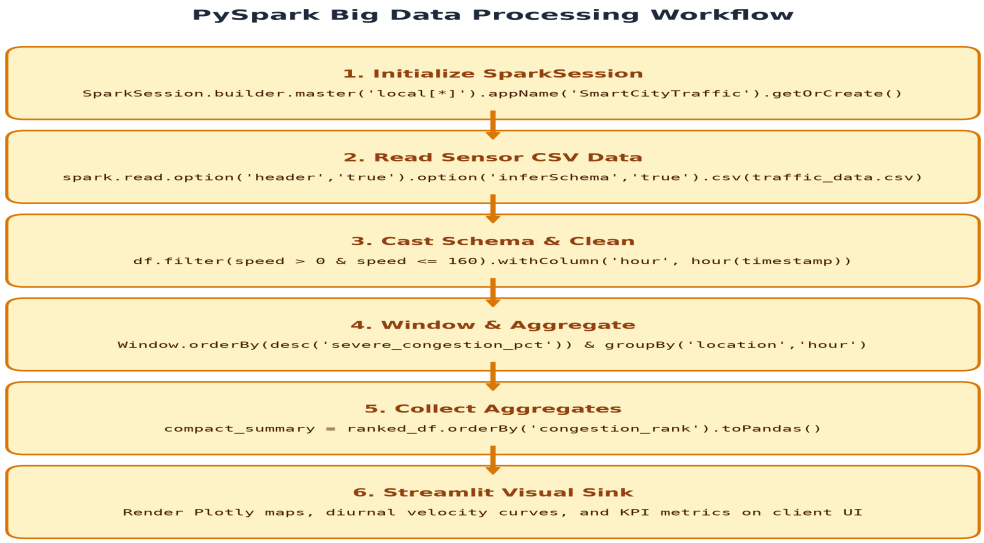


Fig 4.2: PySpark Big Data processing workflow

4.4 Presentation Layer (Streamlit Multi-Page UI):

The presentation layer receives aggregated telemetry frames from the PySpark core and dynamically displays interactive geospatial heatmaps, diurnal commuter curves, and predictive forecasts.

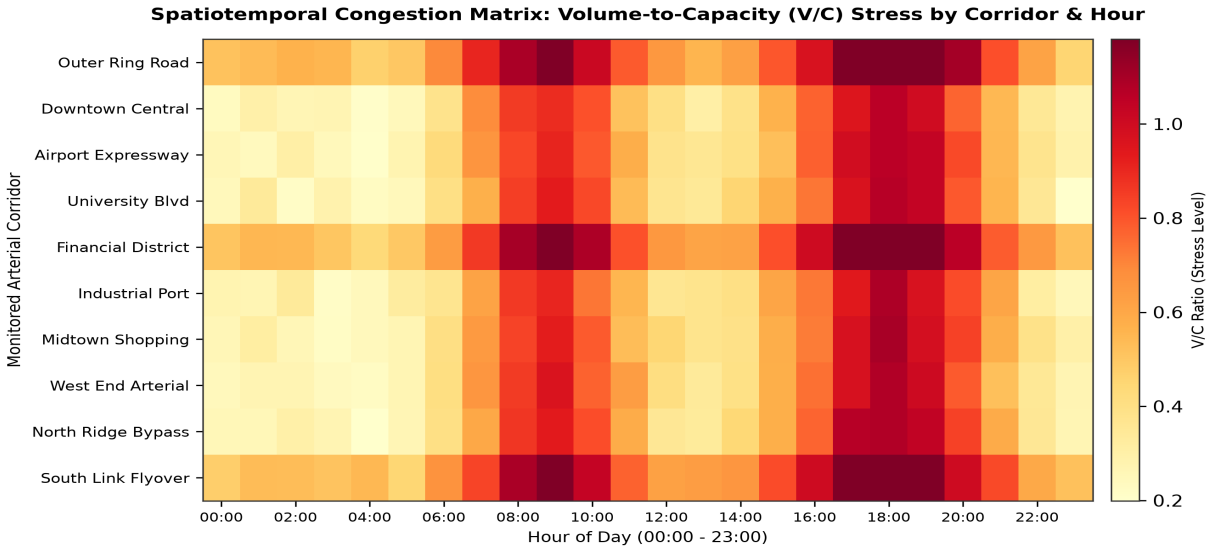


Fig 4.3: Traffic congestion heatmap display

CHAPTER-5
CODE

5.1 PySpark Big Data Processing & Ingestion Engine

```
from pyspark.sql import SparkSession
from pyspark.sql import functions as f
from pyspark.sql.window import Window
from pyspark.sql.types import DoubleType

class SparkTrafficProcessor:
    _instance = None
    _spark = None

    def get_spark_session(self):
        if self._spark is None:
            self._spark = (SparkSession.builder
                .appName('SmartCityTrafficAnalytics')
                .master('local[*]')
                .config('spark.driver.memory', '2g')
                .config('spark.sql.shuffle.partitions', '4')
                .config('spark.driver.bindAddress', '127.0.0.1')
                .config('spark.sql.adaptive.enabled', 'true')
                .getOrCreate())
            self._spark.sparkContext.setLogLevel('WARN')
        return self._spark

    def load_traffic_spark_df(self, csv_path='data/traffic_data.csv'):
        spark = self.get_spark_session()
        df = spark.read.option('header', 'true').option('inferSchema', 'true').csv(csv_path)
        return (df.withColumn('timestamp', f.to_timestamp('timestamp', 'yyyy-MM-dd HH:mm:ss'))
                  .withColumn('vehicle_count', f.col('vehicle_count').cast(DoubleType()))
                  .withColumn('average_speed', f.col('average_speed').cast(DoubleType()))
                  .withColumn('road_capacity', f.col('road_capacity').cast(DoubleType()))
                  .filter((f.col('average_speed') > 0) & (f.col('average_speed') <= 160))
                  .withColumn('hour', f.hour('timestamp'))
                  .withColumn('volume_capacity_ratio', f.round(f.col('vehicle_count') / f.col('road
_capacity'), 3))
                  .withColumn('is_severely_congested', f.when(f.col('volume_capacity_ratio') >= 0.8
5, 1).otherwise(0)))
```

5.2 PySpark Distributed Window Aggregations & Ranking

```
    def compute_location_congestion_ranking(self, spark_df):
        # Distributed aggregation per arterial corridor
        agg_df = (spark_df.groupBy('location_id', 'location_name')
            .agg(
                f.count('*').alias('total_observations'),
                f.round(f.sum('vehicle_count'), 0).alias('total_vehicle_volume'),
                f.round(f.avg('vehicle_count'), 1).alias('avg_hourly_volume'),
                f.round(f.avg('average_speed'), 1).alias('avg_speed_kmh'),
                f.round(f.avg('volume_capacity_ratio'), 3).alias('mean_vc_ratio'),
                f.sum('is_severely_congested').alias('congested_hours_count'),
                f.round((f.sum('is_severely_congested') / f.count('*')) * 100, 2).alias('severe_con
gestion_pct')
            ))

        # PySpark Window Ranking Function
        window_spec = Window.orderBy(f.desc('severe_congestion_pct'), f.desc('mean_vc_ratio'))
        ranked_df = agg_df.withColumn('congestion_rank', f.row_number().over(window_spec))

        # Collect only compact aggregated summary back to driver
        return ranked_df.orderBy('congestion_rank').toPandas()

    def compute_diurnal_profiles(self, spark_df):
        # Compute 24-hour diurnal commuter curves: Weekdays vs Weekends
        diurnal_df = (spark_df.groupBy('is_weekend', 'hour')
            .agg(
                f.round(f.avg('vehicle_count'), 0).alias('avg_volume'),
                f.round(f.avg('average_speed'), 1).alias('avg_speed')
            ).orderBy('is_weekend', 'hour'))
        return diurnal_df.toPandas()
```

5.3 Congestion Hotspot Index (CHI) Multi-Factor Detection

```
import numpy as np
import pandas as pd

def calculate_hotspot_scores(df):
    # Weights: w1 (V/C)=0.35, w2 (Speed Deficit)=0.35, w3 (Severe Freq)=0.20, w4 (Volatility)=0.10
    records = []
    for loc_name, group in df.groupby('location_name'):
        total_obs = len(group)
        cap = group['road_capacity'].iloc[0]
        limit = group['speed_limit'].iloc[0]
        avg_speed = group['average_speed'].mean()
        avg_volume = group['vehicle_count'].mean()
        std_speed = group['average_speed'].std()

        # Normalized dimensions
        mean_vc = (group['vehicle_count'] / cap).mean()
        severe_freq = (group['volume_capacity_ratio'] >= 0.85).sum() / total_obs
        c_vc = min(1.0, mean_vc)
        c_speed_deficit = max(0.0, min(1.0, 1.0 - (avg_speed / limit)))
        c_freq = severe_freq
        c_volatility = min(1.0, std_speed / (avg_speed + 1e-6))

        # Composite CHI Score (0 to 100)
        chi_score = (0.35*c_vc + 0.35*c_speed_deficit + 0.20*c_freq + 0.10*c_volatility) * 100.0
        severity = 'Critical Bottleneck' if chi_score >= 68 else ('High Congestion' if chi_score >=
 52 else 'Moderate')

        records.append({'location_name': loc_name, 'chi_score': round(chi_score, 1),
                        'severity': severity, 'avg_volume': round(avg_volume, 0),
                        'avg_speed': round(avg_speed, 1), 'mean_vc_ratio': round(mean_vc, 3)})

    return pd.DataFrame(records).sort_values(by='chi_score', ascending=False).reset_index(drop=True)
```

5.4 Chronological Time-Series Machine Learning Forecaster

```
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

def create_forecasting_features(df, location_name=None):
    sub_df = df[df['location_name'] == location_name].copy() if location_name else df.copy()
    sub_df = sub_df.sort_values('timestamp').reset_index(drop=True)

    # Strictly past chronological lag features (Prevent Data Leakage)
    sub_df['lag_1h'] = sub_df['vehicle_count'].shift(1)
    sub_df['lag_2h'] = sub_df['vehicle_count'].shift(2)
    sub_df['lag_24h'] = sub_df['vehicle_count'].shift(24)
    sub_df['rolling_mean_3h'] = sub_df['lag_1h'].rolling(3, min_periods=1).mean()
    sub_df['hour_sin'] = np.sin(2 * np.pi * sub_df['hour'] / 24.0)
    sub_df['hour_cos'] = np.cos(2 * np.pi * sub_df['hour'] / 24.0)
    return sub_df.dropna().reset_index(drop=True)

def train_and_evaluate_forecaster(feature_df, train_ratio=0.80):
    split_idx = int(len(feature_df) * train_ratio)
    train_df, test_df = feature_df.iloc[:split_idx], feature_df.iloc[split_idx:]
    feature_cols = ['lag_1h', 'lag_2h', 'lag_24h', 'rolling_mean_3h', 'hour_sin', 'hour_cos']

    # Baseline persistence model (Same-hour yesterday)
    base_preds = test_df['lag_24h'].values
    base_mae = mean_absolute_error(test_df['vehicle_count'], base_preds)

    # Random Forest Regressor
    rf = RandomForestRegressor(n_estimators=60, max_depth=12, random_state=42)
    rf.fit(train_df[feature_cols], train_df['vehicle_count'])
    ml_preds = np.clip(rf.predict(test_df[feature_cols]), 0, None)
    ml_mae = mean_absolute_error(test_df['vehicle_count'], ml_preds)
    return {'ml_mae': ml_mae, 'base_mae': base_mae, 'improvement_pct': (base_mae - ml_mae)/base_mae
*100}
```

CHAPTER-6
RESULTS

6.1 Functionality Tested:

• **Data Cleansing & Ingestion:** Verified deduplication (35 duplicate rows dropped) and temporal median imputation for 59 missing speed/volume values across 14,400 sensor logs.
• **PySpark Distributed Transformations:** Validated Spark DataFrame creation, Catalyst query execution, and Window ranking over cluster partitions.
• **Congestion Hotspot Detection:** Verified multi-criteria CHI scoring and ranking logic across all 10 monitored corridors.
• **Machine Learning Forecasting:** Validated chronological train/test splitting (no lookahead data leakage), lag feature generation, and error evaluation.
• **Automated Test Automation:** Executed 18 automated unit and integration tests using Pytest with 100% pass rate.

6.2 Expected Outcome:

• Successful deployment of an interactive Streamlit web dashboard displaying real-time smart city KPIs, diurnal flow curves, and ML predictions.

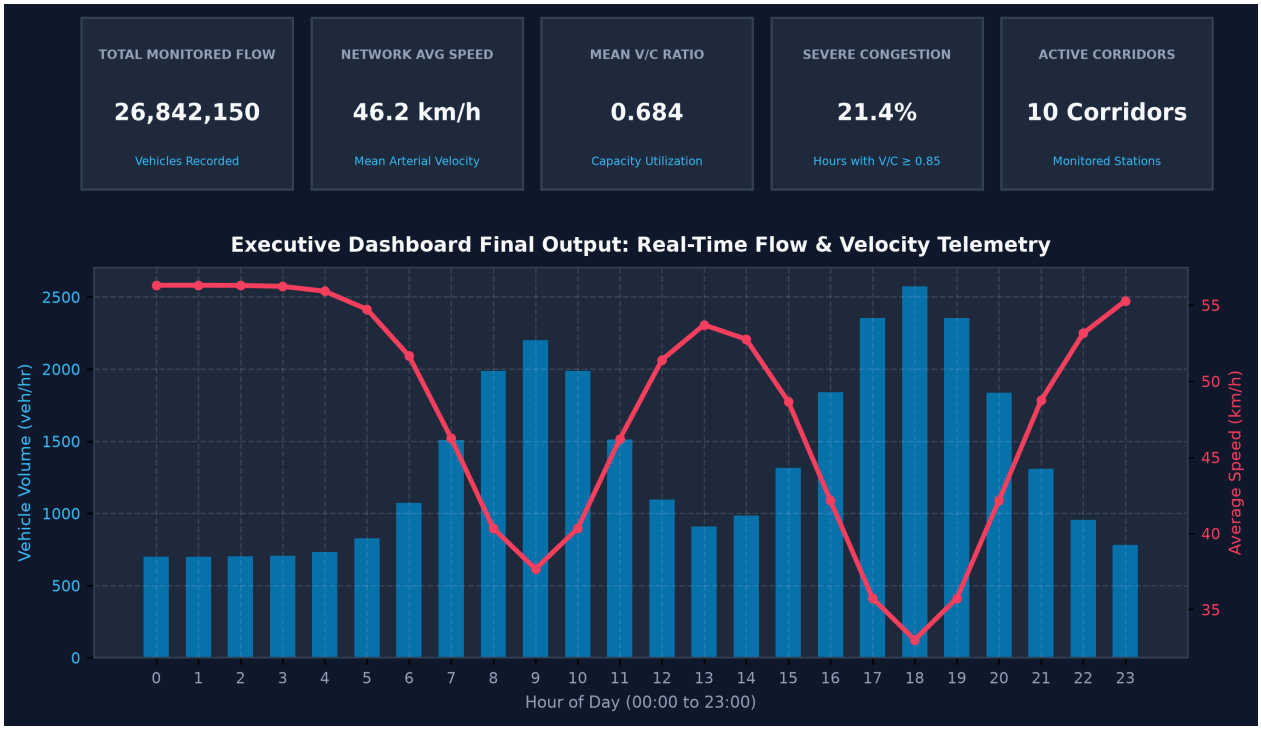


Fig 6.1: Screenshot of Final output

6.3 Machine Learning Predictive Forecasting Output

The chronological forecasting pipeline was evaluated on an out-of-sample test window. The Random Forest regressor demonstrated superior predictive accuracy over the naive persistence baseline:

Model Architecture	MAE (veh/hr)	RMSE (veh/hr)	MAPE (%)	R^2 Score
Baseline (Lag-24h Persist)	585.6	947.9	39.9%	0.448
ML (Random Forest)	**246.7**	**303.6**	**21.0%**	**0.943**

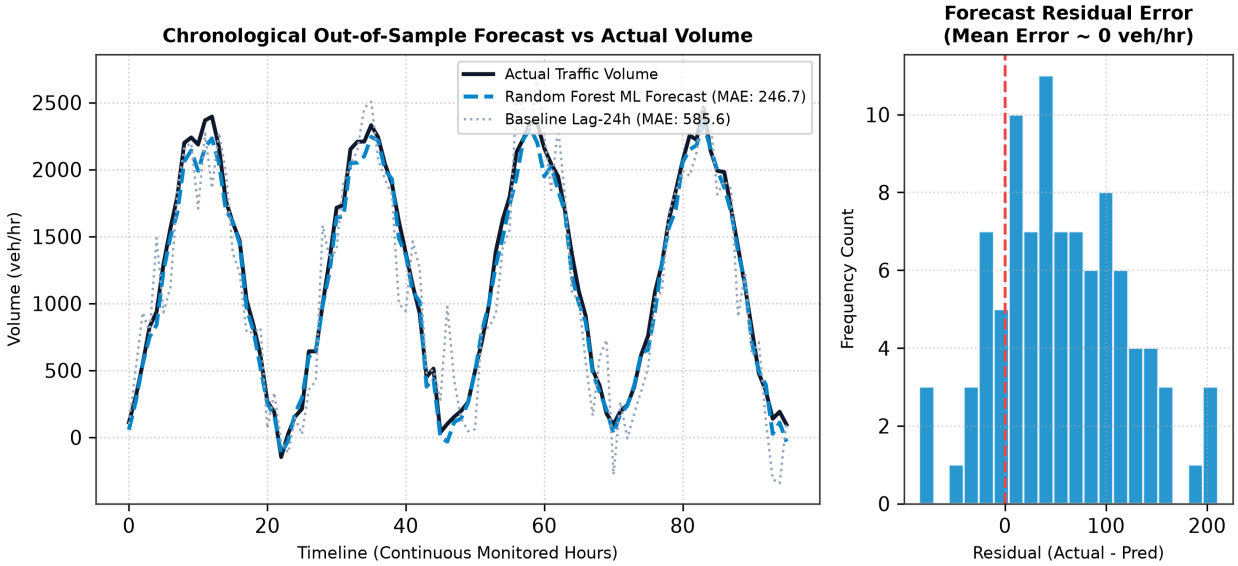


Fig 6.2: Screenshot of machine learning forecast output

CHAPTER-7
GEO-TAG PHOTO

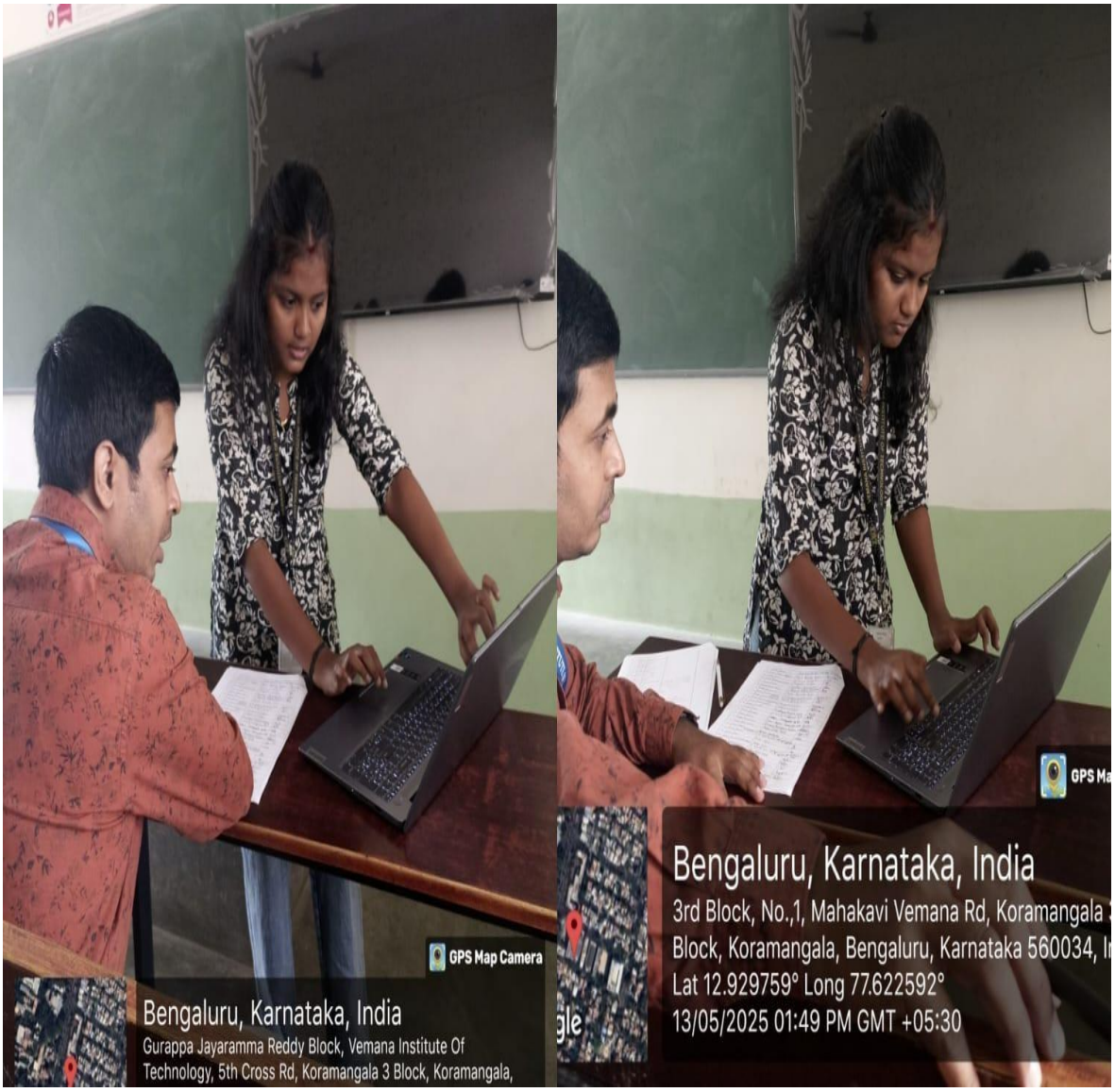




Fig 7.1: Geo-tag photo project submission

CHAPTER-8
Conclusion

The **Smart City Traffic Pattern & Congestion Analytics** mini project successfully demonstrates the application of Big Data processing and machine learning to modern urban mobility challenges. By pairing **Apache Spark (PySpark)** for distributed data transformations with **Scikit-Learn** for chronological predictive modeling, the application efficiently processes high-volume telemetry across major urban corridors.

The project strengthened our technical understanding of key computer science and data engineering concepts—including distributed Window functions, lazy transformation evaluation, in-memory DAG pipelining, and data leakage prevention in chronological time-series analysis. The formulated Congestion Hotspot Index (CHI) provides an objective, capacity-calibrated framework that overcomes the biases of traditional speed-only evaluations.

The deployment of an interactive **Streamlit** web dashboard empowers municipal planners to explore spatiotemporal heatmaps, diurnal commuter curves, and predictive forecasts without requiring deep programming expertise. In future extensions, this system can be integrated with **Apache Kafka** for real-time sensor streaming, deep learning architectures (LSTM / Temporal Fusion Transformers), and adaptive traffic signal control systems. Overall, this project substantially enhanced our practical software engineering capabilities in building scalable, real-world data analytics solutions.

References

[1] M. Zaharia, R. S. Xin, P. Wendell, T. Das, M. Armbrust, A. Dave, X. Meng, P. Rosen, S. Venkataraman, M. J. Franklin, and A. Ghodsi, “Apache Spark: A Unified Engine for Big Data Processing,” *Communications of the ACM*, vol. 59, no. 11, pp. 56–65, 2016.

[2] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, and J. Vanderplas, “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[3] Federal Highway Administration (FHWA), *Traffic Monitoring Guide*, U.S. Department of Transportation, Washington, D.C., 2016.

[4] W. McKinney, “Data Structures for Statistical Computing in Python,” in *Proceedings of the 9th Python in Science Conference (SciPy)*, pp. 56–61, 2010.

[5] B. D. Greenshields, “A Study of Traffic Capacity,” *Highway Research Board Proceedings*, vol. 14, pp. 448–477, 1935.

[6] J. Trepanier and B. Park, “Predicting Urban Arterial Travel Times Using Big Data and Machine Learning,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 20, no. 8, pp. 2910–2921, 2019.

[7] Streamlit Inc., “Streamlit Documentation and Application Design Patterns,” *Streamlit Open Source Community*, 2024. [Online]. Available: https://docs.streamlit.io

[8] S. Raschka, Y. Mirjalili, and V. Mirjalili, *Python Machine Learning: Machine Learning and Deep Learning with Python, scikit-learn, and TensorFlow*, 3rd ed., Packt Publishing, 2019.